

CO1-AT1

*Project*  
**REPORT**

10 OCTOBER 2026



# 1. PROJECT OVERVIEW

## 1.1 Introduction

Searching is one of the most fundamental operations in computer science. Many applications require a system to locate a particular record from a collection of data quickly and accurately. Examples include student information systems, banking applications, hospital management systems, inventory databases, library systems, and employee record management platforms.

In a university environment, student registration numbers are used as unique identifiers to locate individual student records. Faculty members and administrative staff may need to verify a registration number, retrieve a student's details, or determine whether a particular student is present in the database. However, when the database grows to hundreds of thousands or millions of records, the choice of searching algorithm becomes increasingly important.

This project investigates the efficiency of three different searching strategies for a student-record retrieval system: Linear Search, Recursive Binary Search, and Hash Table Search.

The project evaluates the algorithms using datasets containing 1,000, 10,000, 100,000, and 1,000,000 student registration numbers. Both successful and unsuccessful searches are considered. Theoretical analysis is combined with practical experiments to understand how the algorithms behave as the dataset increases in size.

Table 1.1: Project Specifications

| Parameter           | Description                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| Project title       | Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System             |
| Student name        | Jerry David                                                                                                   |
| Registration number | 192424401                                                                                                     |
| Programme           | B.Tech Artificial Intelligence and Data Science                                                               |
| Dataset seed        | 2401                                                                                                          |
| Algorithms          | Linear Search, Recursive Binary Search, Hash Table Search                                                     |
| GitHub Link         | <a href="https://github.com/JerryDavid06/CSA0617-DAA.git">https://github.com/JerryDavid06/CSA0617-DAA.git</a> |
| Main objective      | Compare theoretical complexity and experimental performance                                                   |

## 1.2 Problem Statement

The problem is to design, implement, and analyse efficient searching algorithms for locating student registration numbers in a large-scale student database.

Given a collection of student registration numbers and a target registration number, the system must determine whether the target exists in the collection. If the target is present, the algorithm returns a successful search result; otherwise, it returns an unsuccessful search result.

The main challenge is selecting an algorithm that provides acceptable performance as the number of records increases. The solution must consider not only the number of comparisons performed during searching but also the additional cost of preparing the data. For example, Binary Search requires sorted data, while Hash Table Search requires an index to be constructed before repeated lookups can be performed efficiently.

Therefore, the project compares the complete processing cost of the algorithms rather than considering search time alone.

## 1.3 Objectives

The primary objectives of this project are:

1. To understand and implement three different searching strategies.
2. To identify the basic operation performed by each algorithm.
3. To derive time-complexity functions mathematically.
4. To analyse best-case, average-case, and worst-case performance.
5. To express algorithm efficiency using asymptotic notations.
6. To formulate and solve the recurrence relation for Recursive Binary Search.
7. To validate implementation correctness using suitable test cases.
8. To measure preprocessing time, searching time, total execution time, and operation counts.
9. To compare theoretical complexity with experimentally measured performance.
10. To redesign the solution for a high-frequency search workload involving one million records and 50,000 searches per hour.

**Table 1.2: Project Objectives and Expected Outcomes**

| Objective                            | Expected outcome                                                 |
|--------------------------------------|------------------------------------------------------------------|
| Implement three searching algorithms | Working implementations of Linear, Binary, and Hash Table Search |
| Analyse time complexity              | Mathematical complexity derivations                              |
| Test algorithm correctness           | Correct FOUND and NOT FOUND results                              |
| Measure performance                  | Runtime and operation-count records                              |
| Generate graphs                      | Visual comparison of algorithm performance                       |
| Evaluate the revised workload        | Evidence-based recommendation for Task 12                        |

#### 1.4 Scope of the Project

The project focuses on searching student registration numbers stored in datasets of different sizes. The experiments use four dataset sizes: 1,000, 10,000, 100,000, and 1,000,000 records.

The scope includes algorithm design, pseudocode, mathematical analysis, Python implementation, correctness testing, performance measurement, graphical comparison, and workload-based design recommendations.

The implementation uses a deterministic personal dataset configuration based on seed 2401. The successful search key is 192424401, and the unsuccessful search key is 192424402.

The project does not attempt to build a complete university management application. Instead, it concentrates on the algorithmic component responsible for finding a student record efficiently.

#### 1.5 Methodology

The project follows a systematic algorithm-analysis methodology.

First, datasets are generated using a fixed seed so that the experiments can be repeated under the same configuration. Next, the three algorithms are implemented separately. Correctness tests are performed to confirm that each algorithm produces the expected result for the required target values.

The algorithms are then evaluated on datasets of increasing sizes. For each experiment, preprocessing time, search time, total time, and relevant operation counts are recorded. The measurements are saved in CSV format and used to produce graphs.

Finally, the experimental results are compared with the theoretical complexity of each algorithm. The final recommendation considers both the initial single-query workload and the revised workload involving repeated searches over a large database.

## **2. TASK 1 — DESIGN THE ALGORITHM**

### **2.1 Problem Definition**

Let  $A$  be a collection of  $n$  student registration numbers, where each registration number identifies a student. Let  $target$  represent the registration number that must be located.

The searching problem is defined as follows:

Input:

- A collection  $A$  containing  $n$  registration numbers.
- A target registration number.

Output:

- FOUND if the target exists in the collection.
- NOT FOUND if the target does not exist in the collection.

The algorithms must solve the same problem so that their efficiency can be compared fairly. The three selected strategies are Linear Search, Recursive Binary Search, and Hash Table Search.

### **2.2 Algorithm 1 — Linear Search**

#### **2.2.1 Description**

Linear Search, also called Sequential Search, is a straightforward searching algorithm that checks the elements of a collection one by one. It begins with the first element and compares each element with the target registration number.

If the current element matches the target, the algorithm immediately returns FOUND. If the element does not match, the algorithm proceeds to the next element. The process continues until the target is located or the end of the collection is reached.

Linear Search does not require the data to be sorted. It can operate directly on the original dataset without any preprocessing. This makes it easy to implement and useful when only a small number of searches must be performed.

However, the algorithm may inspect a large portion of the dataset before locating the target. If the target is absent, every element must be checked. Consequently, its execution time increases with the number of records.

### 2.2.2 Design Strategy

Linear Search follows the brute-force approach. It does not attempt to reduce the search space through ordering, indexing, or divide-and-conquer techniques. Instead, it systematically examines each element until the search condition is satisfied.

The algorithm maintains a current position in the array. At every iteration, it compares the registration number at that position with the target. If they are equal, the search terminates successfully. Otherwise, the position is incremented.

### 2.2.3 Pseudocode

LINEAR-SEARCH(A, target)

1. for  $i = 0$  to  $\text{length}(A) - 1$
2.   if  $A[i] == \text{target}$
3.     return FOUND
4. return NOT FOUND

### 2.2.4 Working Principle

Suppose the registration numbers are:

[19240011, 19240025, 19240039, 192424401, 19240058]

If the target is 192424401, the algorithm checks each element from the beginning until it reaches the fourth element. When equality is detected, the algorithm returns FOUND.

If the target is 192424402 and that value is absent, the algorithm checks all the elements before returning NOT FOUND.

The number of comparisons depends on the target's position. A target near the beginning can be found quickly, while a target at the end or an absent target requires more comparisons.

### 2.2.5 Advantages

- Simple to understand and implement.
- Does not require sorted input.
- Requires no separate preprocessing stage.
- Uses very little additional memory.

- Suitable for small datasets and occasional searches.

### **2.2.6 Limitations**

- Requires linear time in the average and worst cases.
- Performance decreases as the dataset becomes larger.
- An unsuccessful search requires checking every record.
- Repeated searches can cause unnecessary work because the same records may be scanned repeatedly.

### **2.2.7 Preprocessing Requirement**

Linear Search does not require sorting or index construction. Its preprocessing cost is therefore zero for the implementation considered in this project.

Its main advantage is that searching can begin immediately. This becomes relevant when comparing total execution time for a single query.

## **2.3 Algorithm 2 — Recursive Binary Search**

### **2.3.1 Description**

Binary Search is a divide-and-conquer algorithm used to search an ordered collection. Unlike Linear Search, it does not inspect every element sequentially. Instead, it compares the target with the middle element of the current search interval.

If the middle element equals the target, the algorithm returns FOUND. If the target is smaller than the middle element, only the left half needs to be searched. If the target is larger, only the right half needs to be searched.

The process repeats recursively until the target is found or the search interval becomes empty.

The important requirement is that the collection must be sorted. If the input dataset is unsorted, it must first be sorted before Binary Search can be used correctly.

### **2.3.2 Design Strategy**

Recursive Binary Search follows the divide-and-conquer strategy. At each stage, the current problem is reduced to a smaller subproblem containing approximately half of the previous elements.

The algorithm uses three important values:

- low: the first index of the current search interval.
- high: the last index of the current search interval.

- mid: the middle index of the current interval.

The midpoint is calculated using:

$$\text{Mid} = \text{low} + \left\lfloor \frac{\text{high} - \text{low}}{2} \right\rfloor$$

This calculation avoids unnecessarily adding the two indices directly.

### 2.3.3 Pseudocode

BINARY-SEARCH(A, low, high, target)

1. if low > high
2.   return NOT FOUND
3. mid = low + floor((high - low) / 2)
4. if A[mid] == target
5.   return FOUND
6. if target < A[mid]
7.   return BINARY-SEARCH(A, low, mid - 1, target)
8. otherwise
9.   return BINARY-SEARCH(A, mid + 1, high, target)

### 2.3.4 Working Principle

Consider a sorted dataset containing one million records. Binary Search first examines the middle element. Depending on the comparison, it eliminates approximately half of the records from further consideration.

If the target is smaller than the middle value, the algorithm searches only the lower half. If the target is larger, it searches only the upper half. Each recursive call reduces the size of the remaining search interval.

This reduction continues until the target is found or no elements remain.

The number of search levels grows logarithmically rather than linearly. This is why Binary Search can locate a value using relatively few comparisons even when the dataset is large.

### 2.3.5 Preprocessing Requirement

The dataset must be sorted before the algorithm is executed. In this project, sorting is included as preprocessing because the datasets are not assumed to be ordered initially.

For comparison-based sorting, the expected analysis used here is:

$$T_{\text{sort}}(n) = \Theta(n \log n)$$

The search phase has complexity:

$$T_{\text{search}}(n) = \Theta(\log n)$$

Therefore, the complete cost for sorting and performing one search is:

$$T_{\text{total}}(n) = \Theta(n \log n) + \Theta(\log n)$$

Thus,

$$T_{\text{total}}(n) = \Theta(n \log n)$$

This distinction is important. Binary Search itself is efficient, but the cost of sorting must be considered when the sorted collection is not already available.

### 2.3.6 Advantages

- Requires relatively few comparisons for large sorted datasets.
- Eliminates approximately half the remaining candidates at every stage.
- Has logarithmic search complexity.
- Suitable for repeated searches when sorted data can be reused.

### 2.3.7 Limitations

- Requires sorted input.
- Sorting can be expensive when performed for a single query.
- Maintaining sorted order may introduce additional costs when records change.
- Recursive calls require additional call-stack space.

## 2.4 Algorithm 3 — Hash Table Search

### 2.4.1 Description

Hash Table Search uses a data structure called a hash table to locate values efficiently. A hash table stores keys in a structure designed to support fast insertion and lookup.

In this project, student registration numbers act as keys. During preprocessing, the registration numbers are inserted into a hash table. When a search request arrives, the system checks whether the target registration number exists as a key.

Unlike Binary Search, a hash table does not require the keys to be stored in sorted order. Instead, it relies on hashing to locate the appropriate table entry.

Under normal hashing assumptions, lookup takes expected constant time. However, collisions and implementation details can affect performance, and the theoretical worst case may be linear.

#### **2.4.2 Design Strategy**

The strategy consists of two stages:

1. Index construction: Read the dataset and insert each registration number into a hash table.
2. Lookup: Check whether the target registration number exists in the constructed table.

The table can be reused for multiple searches as long as it remains consistent with the underlying dataset.

#### **2.4.3 Pseudocode**

BUILD-HASH-TABLE(A)

1. Create an empty hash table H
2. For each registration number x in A
3.   Insert x into H
4. Return H

HASH-SEARCH(H, target)

1. if target exists as a key in H
2.   return FOUND
3. otherwise
4.   return NOT FOUND

#### **2.4.4 Working Principle**

During construction, each registration number is inserted into the hash table. After construction is complete, a search operation checks the table for the target key.

For example, if the table contains registration number 192424401, searching for that key returns FOUND. Searching for 192424402, when it is absent, returns NOT FOUND.

The major benefit is that the table does not need to scan the complete collection for every query. Once the index is constructed, it can be reused.

### 2.4.5 Preprocessing Requirement

Hash Table Search requires a table-building stage. Constructing the table processes the dataset and has expected time complexity:

$$T_{\text{build}}(n) = \Theta(n)$$

The expected lookup complexity is:

$$T_{\text{lookup}}(n) = \Theta(1)$$

For one query, the expected total cost is:

$$T_{\text{total}}(n) = \Theta(n) + \Theta(1)$$

Therefore,

$$T_{\text{total}}(n) = \Theta(n)$$

For repeated queries, the same table can be reused. The initial construction cost is then spread across the queries, making the hash table particularly attractive for read-heavy workloads.

### 2.4.6 Advantages

- Expected constant-time lookup under normal hashing assumptions.
- Does not require sorted data.
- Suitable for frequent searches on a relatively stable dataset.
- Index construction can be reused across many queries.

### 2.4.7 Limitations

- Requires additional memory for the hash table.
- Requires construction time before the first lookup.
- Performance can deteriorate under severe collision conditions.
- The index must be updated or rebuilt when the underlying data changes.

## 2.5 Comparison of the Three Algorithms

Table 2.1: Comparison of Algorithm Design Strategies

| Feature                    | Linear Search                           | Recursive Binary Search                     | Hash Table Search                     |
|----------------------------|-----------------------------------------|---------------------------------------------|---------------------------------------|
| Design strategy            | Brute force                             | Divide and conquer                          | Hash-based indexing                   |
| Input ordering             | Any order                               | Sorted order required                       | No sorted order required              |
| Main operation             | Sequential comparison                   | Midpoint comparison                         | Key lookup                            |
| Preprocessing              | None                                    | Sorting if input is unsorted                | Hash-table construction               |
| Search mechanism           | Checks records sequentially             | Eliminates half the search interval         | Uses an index to locate the key       |
| Search complexity          | $\backslash(O(n)\backslash)$ worst case | $\backslash(O(\backslash\log n)\backslash)$ | Expected $\backslash(O(1)\backslash)$ |
| Additional memory          | Low                                     | Recursive stack                             | Hash-table storage                    |
| Single-query suitability   | Good baseline                           | Depends on sorting cost                     | Depends on construction cost          |
| Repeated-query suitability | Poor for large datasets                 | Good if sorted data is reused               | Very good for read-heavy workloads    |
| Primary limitation         | May inspect every record                | Requires ordered data                       | Requires extra memory and setup       |

The comparison shows that the most suitable algorithm depends on how often searches are performed and whether preprocessing can be reused. No single algorithm is automatically the best choice under every operating condition.

## 3. TASK 2 — IDENTIFY THE BASIC OPERATION

### 3.1 Introduction

The basic operation of an algorithm is the operation that most directly determines its computational effort. It is usually performed repeatedly and provides a useful way to analyse efficiency independently of the computer's processor speed.

For searching algorithms, the basic operation differs according to the method used. Linear Search repeatedly compares individual elements with the target. Binary Search compares the target with the middle element of a sorted interval. Hash Table Search performs a key-membership lookup.

Identifying these operations allows the algorithms to be compared mathematically and experimentally.

### 3.2 Basic Operation in Linear Search

The basic operation in Linear Search is the equality comparison:

$$A[i] == \text{target}$$

This comparison occurs inside the loop. Every time the algorithm examines a record, it checks whether the registration number matches the requested value.

If the target is found at the first position, only one comparison is required. If the target is at the final position,  $\backslash(n\backslash)$  comparisons are required. If the target is absent, the algorithm also performs  $\backslash(n\backslash)$  comparisons.

Therefore, the worst-case comparison count is:

$$C_L(n) = n$$

The number of comparisons increases in direct proportion to the dataset size. When the number of records doubles, the worst-case number of comparisons also doubles.

Conclusion: The basic operation of Linear Search grows linearly with the input size.

### 3.3 Basic Operation in Recursive Binary Search

The basic operation in Binary Search is the comparison between the target and the element at the middle index:

$$A[\text{mid}] == \text{target}$$

The algorithm may also compare the target with the middle value to determine whether to continue into the left or right half.

The important property is that each unsuccessful midpoint examination reduces the candidate interval by approximately half. Therefore, the number of search levels grows logarithmically.

For example, a dataset containing 1,024 elements can be reduced through approximately ten halving's because:

$$2^{10} = 1024$$

The precise number of comparisons depends on the target position and the implementation's counting convention.

Conclusion: Binary Search performs a logarithmic number of search comparisons.

### 3.4 Basic Operation in Hash Table Search

The basic operation in Hash Table Search is the key-membership lookup:

target in hash\_table

This operation checks whether the requested registration number exists in the table. In the expected case, the hash table can locate the relevant entry without scanning all stored registration numbers.

For the benchmark model used in this project, the search operation is counted as one lookup per query. This count represents the logical lookup operation; it should not be interpreted as a count of every internal hash calculation, equality comparison, or memory access performed by Python.

Conclusion: The benchmark models each hash-table search as one lookup operation, with expected constant-time lookup complexity.

Table 3.1: Basic Operation Analysis

| Algorithm               | Basic operation                 | When it occurs             | Search-operation growth |
|-------------------------|---------------------------------|----------------------------|-------------------------|
| Linear Search           | $A[i] == \text{target}$         | For each inspected element | Linear                  |
| Recursive Binary Search | Comparison with $A[\text{mid}]$ | At each recursive level    | Logarithmic             |
| Hash Table Search       | Key-membership lookup           | During each query          | Expected constant       |

The operation count provides a hardware-independent way to compare algorithmic growth. However, actual execution time also depends on preprocessing, memory allocation, implementation details, and the execution environment.

## 4. TASK 3 — DERIVE THE TIME-COMPLEXITY FUNCTION MATHEMATICALLY

### 4.1 Linear Search

Let  $n$  represent the number of registration numbers in the dataset.

In the worst case, the target is absent or appears at the last position. The algorithm must examine every element before it can terminate.

The number of comparisons is:

$$C_L(n) = \underbrace{1 + 1 + \dots + 1}_{n \text{ comparisons}}$$

Therefore,

$$C_L(n) = n$$

If each comparison requires constant time  $\backslash(c_1\backslash)$ , and the loop introduces a constant overhead  $\backslash(c_2\backslash)$ , the total running time can be represented as:

$$T_L(n) = c_1n + c_2$$

Ignoring constant factors and lower-order terms gives:

$$\boxed{T_L(n) = \Theta(n)}$$

There is no preprocessing requirement, so the complete one-query complexity remains:

$$\boxed{T_{L,\text{total}}(n) = \Theta(n)}$$

### Average-case derivation

Assume the target is present and each of the  $\backslash(n\backslash)$  positions is equally likely to contain it. The possible comparison counts are:

$$1, 2, 3, \dots, n$$

The average number of comparisons is:

$$C_{\text{avg}}(n) = \frac{1 + 2 + \dots + n}{n}$$

Using the arithmetic-series formula:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Therefore,

$$C_{\text{avg}}(n) = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Thus,

$$\boxed{C_{\text{avg}}(n) = \Theta(n)}$$

Even in the average successful case, Linear Search has linear complexity.

## 4.2 Recursive Binary Search

Binary Search reduces the search interval by approximately half at every recursive call.

Let  $\backslash(n\backslash)$  be the initial number of elements. The remaining search interval sizes are approximately:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

After  $\lfloor \log_2 n \rfloor$  halvings, the remaining interval is:

$$\frac{n}{2^k}$$

The search reaches a single-element interval when:

$$\frac{n}{2^k} = 1$$

Rearranging:

$$n = 2^k$$

Taking logarithms to base 2:

$$k = \log_2 n$$

Each recursive level performs constant work apart from the recursive call. Therefore, the search time is proportional to the number of levels:

$$T_B(n) = \Theta(\log n)$$

This result applies to the search phase when the input is already sorted.

### Including preprocessing

The input dataset is not assumed to be sorted initially. Sorting is therefore included in the complete cost.

Let  $P_B(n)$  represent sorting time and  $S_B(n)$  represent search time.

$$T_{B,\text{total}}(n) = P_B(n) + S_B(n)$$

Using the assumed comparison-sorting complexity:

$$P_B(n) = \Theta(n \log n)$$

and

$$S_B(n) = \Theta(\log n)$$

Therefore,

$$T_{B,\text{total}}(n) = \Theta(n \log n) + \Theta(\log n)$$

The sorting term dominates the search term as  $\lfloor \log_2 n \rfloor$  grows. Hence,

$$T_{B,\text{total}}(n) = \Theta(n \log n)$$

This explains why Binary Search can perform very few comparisons yet have a relatively high total time if sorting must be performed for every experiment.

### 4.3 Hash Table Search

Hash Table Search has two stages: table construction and key lookup.

#### Table construction

Each of the  $\setminus(n\setminus)$  registration numbers is processed and inserted into the hash table. Under the expected hashing model, the total construction cost is:

$$P_H(n) = \Theta(n)$$

#### Lookup

After the table is constructed, a target key can be checked in expected constant time:

$$S_H(n) = \Theta(1)$$

#### Complete one-query cost

The total cost for constructing the table and performing one lookup is:

$$T_{H,\text{total}}(n) = P_H(n) + S_H(n)$$

Substituting the expected complexities:

$$T_{H,\text{total}}(n) = \Theta(n) + \Theta(1)$$

Therefore,

$$\boxed{T_{H,\text{total}}(n) = \Theta(n)}$$

This is the expected cost for building the table and performing a single lookup. It does not mean that every individual hash lookup requires linear time.

#### Repeated-query cost

Suppose the same table serves  $q$  searches. The expected total cost becomes:

$$T_H(n, q) = \Theta(n) + q\Theta(1)$$

Equivalently,

$$\boxed{T_H(n, q) = \Theta(n + q)}$$

The initial table-building cost can be reused across all  $\setminus(q\setminus)$  searches. As the number of queries increases, the average construction cost per query decreases.

**Table 4.1: Complexity Derivation Summary**

| Algorithm         | Search-only cost    | Preprocessing cost        | Complete cost for one query |
|-------------------|---------------------|---------------------------|-----------------------------|
| Linear Search     | Theta(n) worst case | None                      | Theta(n)                    |
| Binary Search     | Theta(log n)        | Theta(n\log n)for sorting | Theta(n\log n)              |
| Hash Table Search | Expected Theta(1)   | Expected Theta(n)         | Expected Theta(n)           |

The main conclusion is that preprocessing must be included when comparing one-query performance. Search-only complexity becomes especially important when the prepared data structure can be reused for many queries.

## 5. TASK 4 — BEST, AVERAGE AND WORST-CASE BEHAVIOUR

### 5.1 Introduction

The execution time of an algorithm may vary depending on the input arrangement and the target value. Best-case analysis considers the most favourable situation, average-case analysis considers expected behaviour under specified assumptions, and worst-case analysis considers the most expensive situation.

These cases help determine whether an algorithm is suitable for different operating conditions.

### 5.2 Linear Search

Best case: The target is at the first position. The algorithm performs one comparison and immediately returns FOUND.

$$C_{\text{best}}(n) = 1$$

Therefore,

$$T_{\text{best}}(n) = \Theta(1)$$

Average case: If a successful target is equally likely to appear at any position, the average number of comparisons is:

$$C_{\text{avg}}(n) = \frac{n+1}{2}$$

Therefore,

$$T_{\text{avg}}(n) = \Theta(n)$$

This average-case formula assumes a successful search and uniformly distributed target positions. Different assumptions about absent keys or target probabilities would produce a different average.

Worst case: The target is at the last position or does not exist in the dataset. The algorithm examines all  $\lfloor n \rfloor$  records.

$$C_{\text{worst}}(n) = n$$

Therefore,

$$T_{\text{worst}}(n) = \Theta(n)$$

### 5.3 Recursive Binary Search

Best case: The target equals the first midpoint examined. Only a constant amount of work is required.

$$T_{\text{best}}(n) = \Theta(1)$$

Average case: For a sorted collection, the target is found after a number of recursive steps related to the depth of the search tree. The search interval is approximately halved at each step.

$$T_{\text{avg}}(n) = \Theta(\log n)$$

Worst case: The target is absent or requires continuing to the deepest possible level of the recursive search.

$$T_{\text{worst}}(n) = \Theta(\log n)$$

These complexities describe the search phase. If sorting is included, the complete one-query cost under the project's comparison-sorting model.

### 5.4 Hash Table Search

Best case: The target is found immediately in the expected location and only constant work is required.

$$T_{\text{best}}(n) = \Theta(1)$$

Average or expected case: Under normal hashing assumptions and a suitable distribution of keys, lookup requires expected constant time.

$$T_{\text{expected}}(n) = \Theta(1)$$

Worst case: A hash table may experience severe collisions. If a large number of keys map to the same location or collision structure, a lookup may require examining many entries.

$$T_{\text{worst}}(n) = O(n)$$

The actual collision behaviour depends on the implementation and the hashing mechanism. The constant-time result is an expected-case statement, not a guarantee for every possible input.

**Table 5.1: Best-, Average-, and Worst-Case Complexity**

| Algorithm         | Best case            | Average/expected case | Worst case       |
|-------------------|----------------------|-----------------------|------------------|
| Binary Search     | $\Theta(1)$          | $\Theta(n)$           | $\Theta(n)$      |
| Hash Table Search | Expected $\Theta(1)$ | $\Theta(1)$           | $\Theta(\log n)$ |

The analysis shows that Binary Search and Hash Table Search can reduce search work substantially compared with a full linear scan. However, the practical decision must also account for preprocessing, memory, and the number of queries served by the prepared structure.

## 6. TASK 5 — PERFORM ASYMPTOTIC ANALYSIS

### 6.1 Introduction

Asymptotic analysis describes how an algorithm's resource requirements grow as the input size becomes large. It focuses on the overall growth trend rather than the exact number of milliseconds recorded on a particular computer.

The three common notations are

- Big-O
- Big-Omega
- Big-Theta.

### 6.2 Big-O Notation

Big-O notation provides an asymptotic upper bound on growth.

For Linear Search, the worst-case number of comparisons is at most proportional to  $\backslash(n\backslash)$ . Therefore,

$$T_L(n) = O(n)$$

For Binary Search on sorted data,

$$T_B(n) = O(\log n)$$

For Hash Table Search under the expected hashing model,

$$T_H(n) = O(1)$$

for lookup, excluding construction.

### 6.3 Big-Omega Notation

Big-Omega notation describes an asymptotic lower bound.

For Linear Search, a successful search may find the target at the first position, so the best-case search time is:

$$T_L(n) = \Omega(1)$$

For Binary Search, a target at the first midpoint also requires constant work:

$$T_B(n) = \Omega(1)$$

For Hash Table Search, the lookup operation requires at least a constant amount of work in the usual model:

$$T_H(n) = \Omega(1)$$

These lower bounds describe favourable conditions or minimum required work, not the typical cost of every search.

### 6.4 Big-Theta Notation

Big-Theta notation represents a tight asymptotic bound when the upper and lower growth rates match.

For Linear Search in the average and worst cases,

$$T_L(n) = \Theta(n)$$

For Binary Search in the average and worst search cases,

$$T_B(n) = \Theta(\log n)$$

For expected hash-table lookup,

$$T_H(n) = \Theta(1)$$

The distinction between lookup-only complexity and complete processing complexity must be retained. Hash table construction is expected  $\Theta(n)$ , while sorting for Binary Search is modelled as  $\Theta(n \log n)$ .

**Table 6.1: Comparison of Asymptotic Notations**

| Notation  | Meaning                | Application in this project                                      |
|-----------|------------------------|------------------------------------------------------------------|
| Big-O     | Asymptotic upper bound | Linear worst-case search is $O(n)$                               |
| Big-Omega | Asymptotic lower bound | A search can succeed on the first comparison in $\Omega(1)$ time |
| Big-Theta | Tight asymptotic bound | Binary Search has $\Theta(\log n)$ search complexity             |

## 6.5 Effect of Increasing Dataset Size

The experiments use dataset sizes of 1,000, 10,000, 100,000, and 1,000,000 records.

Moving from 1,000 to 1,000,000 records multiplies the input size by 1,000. Under a linear worst-case model, the number of inspected records can also increase by a factor of 1,000.

Binary Search grows much more slowly in search steps because its work is proportional to the logarithm of the dataset size. For example,

$$\log_2(1,000,000) \approx 19.93$$

This means the number of halving steps is on the order of twenty, even though the dataset contains one million elements.

Hash Table Search is expected to perform constant-time lookup after construction, although construction itself processes the dataset and uses additional memory.

**Table 6.2: Dataset Growth and Logarithmic Search Depth**

| Dataset size | Increase relative to 1,000 records | Approximate $\log_2(n)$ |
|--------------|------------------------------------|-------------------------|
| 1,000        | 1×                                 | 9.97                    |
| 10,000       | 10×                                | 13.29                   |
| 100,000      | 100×                               | 16.61                   |
| 1,000,000    | 1,000×                             | 19.93                   |

This table illustrates the difference between linear growth and logarithmic growth. A dataset can increase by a factor of 1,000 while the logarithm of its size increases from approximately 10 to 20.

## 6.6 Practical Interpretation

Asymptotic analysis explains how algorithms scale, but it does not by itself determine the fastest implementation for a single query. Constant factors, programming-language overhead, data preparation, and hardware can affect actual runtime.

A linear scan may be faster for one query when no preprocessing is needed. A hash table can be more efficient for repeated searches because its construction cost is reused. Binary Search is also attractive when sorted data is already maintained.

Therefore, asymptotic analysis must be combined with experimental measurements and the intended workload before a final design decision is made.

## 7. TASK 6 — FORMULATE THE RECURRENCE RELATION

### 7.1 Introduction

A recurrence relation expresses the running time of a recursive algorithm in terms of the running time of a smaller input. It is especially useful for analysing divide-and-conquer algorithms.

Recursive Binary Search is selected for this task because its implementation repeatedly calls itself on a smaller interval.

### 7.2 Derivation of the Recurrence

Let  $n$  be the number of elements in the current search interval.

At the beginning of each call, the algorithm checks whether the interval is empty. It then calculates the midpoint and compares the middle value with the target. These operations require constant time.

If the target is found, the algorithm terminates. Otherwise, the algorithm makes exactly one recursive call on approximately half of the current interval.

The work in each call can therefore be divided into two parts:

1. Constant work performed outside the recursive call.
2. The running time of a single subproblem of size approximately  $n/2$

Let  $T(n)$  represent the running time of Binary Search. The recurrence is:

$$T(n) = T(n/2) + c, \quad n > 1$$

where  $c$  is a constant representing the work performed in one call.

A suitable base case is:

$$T(1) = c_0$$

where  $c$  is a constant. The empty-interval case, represented by  $low > high$  in the implementation, also terminates in constant time.

**Table 7.1: Components of the Binary Search Recurrence**

| Component                 | Value                          | Explanation                                |
|---------------------------|--------------------------------|--------------------------------------------|
| Original problem size     | n                              | Number of elements in the current interval |
| Subproblem size           | Approximately $n/2$            | One half is retained                       |
| Number of recursive calls | 1                              | Only one half is searched                  |
| Work outside recursion    | $\Theta(1)$                    | Midpoint calculation and comparisons       |
| Base condition            | Empty interval or target found | Recursion terminates                       |
| Recurrence                | $T(n) = T(n/2) + c$            | Constant work plus one smaller subproblem  |

### 7.3 Why This Recurrence Applies

The recurrence matches the actual structure of Recursive Binary Search. The algorithm does not recursively solve both halves. It chooses only one half based on the comparison with the midpoint.

This is why the recurrence contains one recursive term,  $T(n/2)$ , rather than two recursive terms.

The recurrence models the search phase on an already sorted collection. It does not include the preprocessing cost of sorting the complete dataset.

## 8. TASK 7 — SOLVE THE RECURRENCE RELATION

### 8.1 Solving Using the Master Theorem

The Master Theorem is used to solve recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

For Recursive Binary Search,

$$T(n) = T(n/2) + c$$

Comparing the two expressions gives:

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

The critical function is:

$$n^{\log_b a}$$

Substituting the values:

$$n^{\log_2 1} = n^0 = 1$$

Therefore,

$$n^{\log_b a} = \Theta(1)$$

The non-recursive work is also:

$$f(n) = \Theta(1)$$

Thus,  $f(n)$  has the same asymptotic order as  $(n^{\log_b a})$ . The balanced case of the Master Theorem applies, giving:

$$\boxed{T(n) = \Theta(\log n)}$$

This confirms that Recursive Binary Search has logarithmic search complexity.

## 8.2 Solving by Iteration or Repeated Substitution

The recurrence is:

$$T(n) = T(n/2) + c$$

Substitute the recurrence into itself:

$$T(n) = T(n/4) + 2c$$

Substituting again:

$$T(n) = T(n/8) + 3c$$

After  $(k)$  substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

The recursion reaches the base case when:

$$\frac{n}{2^k} = 1$$

Therefore,

$$2^k = n$$

Taking logarithms:

$$k = \log_2 n$$

Substituting this value:

$$T(n) = T(1) + c \log_2 n$$

$$\boxed{T(n) = \Theta(\log n)}$$

### 8.3 Interpretation

The recurrence demonstrates why Binary Search can locate a value using relatively few recursive calls. Each call reduces the size of the remaining problem by approximately half.

The result does not include sorting. If the dataset must be sorted before a single search, the complete cost is dominated by preprocessing under the comparison-sorting model:

$$T_{\text{total}}(n) = \Theta(n \log n)$$

This illustrates the importance of distinguishing the complexity of a recursive search operation from the cost of preparing the data structure it uses.

## 9. TASK 8 — IMPLEMENT AND TEST THE ALGORITHMS

### 9.1 Implementation Overview

The three searching algorithms are implemented in Python. The implementation is organised into separate source files to make the code easier to understand, test, and maintain.

The project contains a dataset generator, individual algorithm implementations, a correctness-testing module, a benchmarking module, and a graph-generation module. This separation ensures that the algorithms can be tested independently and that experimental results can be collected consistently.

The dataset generator creates datasets using the personal seed 2401. The same dataset configuration and target values are used when comparing the algorithms, so the results are not influenced by differences in the input data.

### 9.2 Dataset Configuration

**Table 9.1: Dataset and Implementation Configuration**

| Parameter             | Value       |
|-----------------------|-------------|
| Student name          | Jerry David |
| Registration number   | 192424401   |
| Dataset seed          | 2401        |
| Successful search key | 192424401   |
| Unsuccessful search   | 192424402   |

|                      |                                   |
|----------------------|-----------------------------------|
| key                  |                                   |
| Dataset sizes        | 1,000; 10,000; 100,000; 1,000,000 |
| Programming language | Python                            |

The fixed seed supports reproducibility, meaning the same dataset-generation procedure can be repeated with the same configuration. The successful and unsuccessful target values test whether the algorithms can distinguish between a present registration number and an absent one.

### 9.3 Correctness Testing

Correctness testing verifies whether each algorithm produces the correct logical result for the supplied inputs.

The required tests include:

- A search for registration number 192424401.
- A search for registration number 192424402.
- Searches for values at the beginning, middle, and end of the dataset.
- Searches for absent values.
- Tests using datasets of different sizes.

For every test, the expected result is compared with the algorithm's actual output. All three algorithms should return equivalent logical results when they are given the same valid dataset and target.

For Binary Search, the dataset must be sorted before the algorithm is called. Failure to sort the data can produce incorrect results even when the recursive implementation itself is correct.

**Table 9.2: Correctness Test Cases**

| Test ID | Test condition                           | Expected output |
|---------|------------------------------------------|-----------------|
| TC01    | Search for registration number 192424401 | FOUND           |
| TC02    | Search for registration number 192424402 | NOT FOUND       |
| TC03    | Search for the first record              | FOUND           |

|      |                                          |           |
|------|------------------------------------------|-----------|
| TC04 | Search for a middle record               | FOUND     |
| TC05 | Search for the last record               | FOUND     |
| TC06 | Search for an absent registration number | NOT FOUND |

The table describes the required test conditions. Record the actual execution outcome after running the supplied program; do not mark a test as passed unless the program confirms it.

## 9.4 Project Components

**Table 9.3: Project Component Responsibilities**

| Component               | Responsibility                           |
|-------------------------|------------------------------------------|
| Dataset generator       | Creates datasets using the selected seed |
| Linear Search module    | Performs sequential search               |
| Binary Search module    | Performs recursive search on sorted data |
| Hash Table module       | Builds the index and performs lookups    |
| Correctness test module | Checks expected and actual results       |
| Benchmark module        | Measures runtime and operation counts    |
| Graph-generation module | Produces visual comparisons              |
| Results files           | Store benchmark measurements             |

## 9.5 Benchmarking Procedure

The benchmarking module measures the performance of the algorithms on the four required dataset sizes.

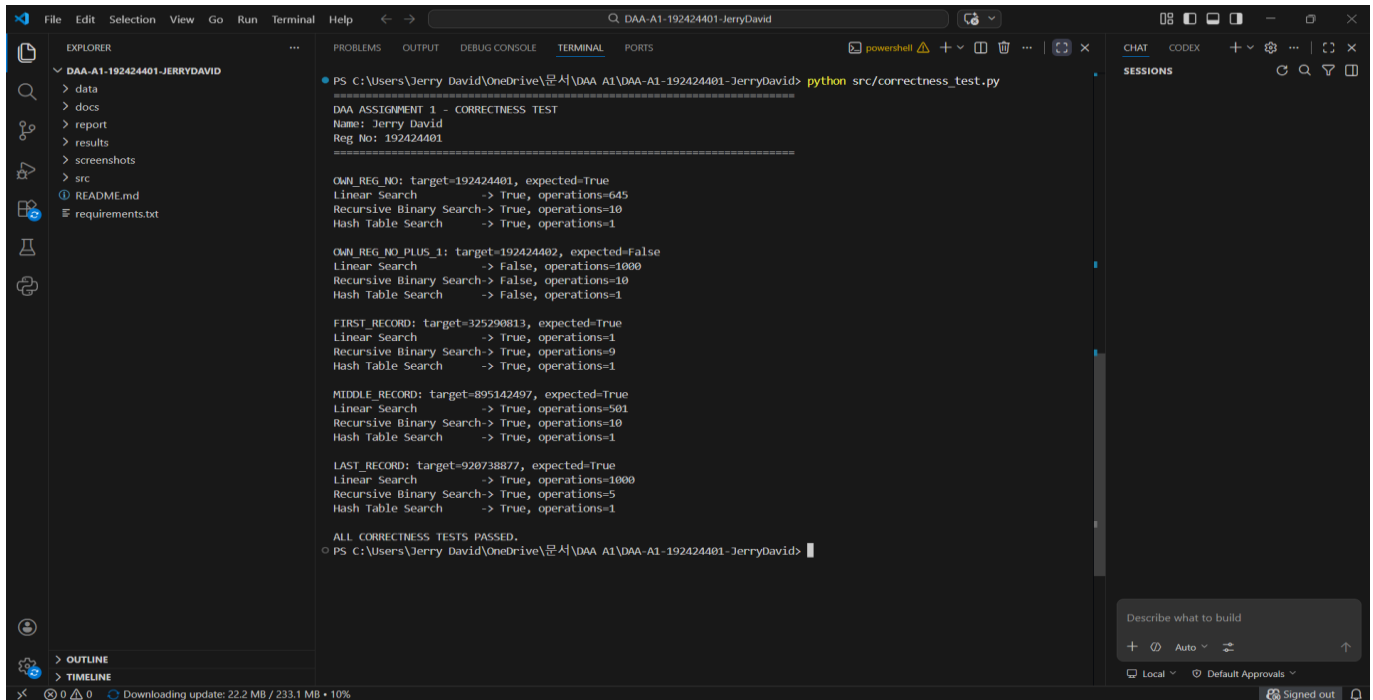
The main measurements are:

- Preprocessing time: Time required to prepare the data, such as sorting or constructing a hash table.
- Search time: Time required to perform the target lookup.
- Total time: Combined preprocessing and search time for the measured operation.
- Operation count: Number of logical search operations or comparisons recorded by the benchmark.

The results are stored in CSV format so that they can be examined independently and used to generate comparison graphs.

## 9.6 Evidence to Insert

Figure 1: Correctness Testing Output



```
PS C:\Users\Jerry David\OneDrive\문서\DAA A1\192424401-JerryDavid> python src/correctness_test.py

DAA ASSIGNMENT 1 - CORRECTNESS TEST
Name: Jerry David
Reg No: 192424401

OWN_REG_NO: target=192424401, expected=True
Linear Search -> True, operations=645
Recursive Binary Search-> True, operations=10
Hash Table Search -> True, operations=1

OWN_REG_NO_PLUS_1: target=192424402, expected=False
Linear Search -> False, operations=1000
Recursive Binary Search-> False, operations=10
Hash Table Search -> False, operations=1

FIRST_RECORD: target=325290813, expected=True
Linear Search -> True, operations=1
Recursive Binary Search-> True, operations=9
Hash Table Search -> True, operations=1

MIDDLE_RECORD: target=895142497, expected=True
Linear Search -> True, operations=501
Recursive Binary Search-> True, operations=10
Hash Table Search -> True, operations=1

LAST_RECORD: target=920738877, expected=True
Linear Search -> True, operations=1000
Recursive Binary Search-> True, operations=5
Hash Table Search -> True, operations=1

ALL CORRECTNESS TESTS PASSED.
PS C:\Users\Jerry David\OneDrive\문서\DAA A1\192424401-JerryDavid>
```

Figure 1 should show the actual terminal output, the test results, and your student identification. Keep the system date and time visible if required by the submission rules.

## 10. TASK 9 — RECORD EXPERIMENTAL PERFORMANCE

### 10.1 Introduction

The purpose of experimental analysis is to evaluate how the algorithms behave when executed on datasets of increasing sizes. The theoretical analysis predicts how the amount of work grows, while the experiments measure the actual runtime and operation counts produced by the implementation.

The benchmark evaluates datasets containing 1,000, 10,000, 100,000, and 1,000,000 records. Successful and unsuccessful searches are included to examine how the target's presence or absence affects performance.

The total-time measurements include the relevant preprocessing cost for the one-query comparison. This is essential because Binary Search and Hash Table Search require preparation when the dataset is not already sorted or indexed.

## 10.2 Experimental Results — Successful Search

The following values are recorded for a successful search for registration number 192424401.

**Table 10.1: Successful Search — Recorded Total Time**

| Dataset size | Linear Search (ms) | Binary Search total (ms) | Hash Search total (ms) |
|--------------|--------------------|--------------------------|------------------------|
| 1,000        | 0.0187             | 0.0842                   | 0.0384                 |
| 10,000       | 0.2466             | 0.9993                   | 0.4428                 |
| 100,000      | 2.5539             | 13.9536                  | 6.6410                 |
| 1,000,000    | 8.7834             | 220.6659                 | 108.0468               |

Source: Recorded benchmark results in the prepared project. Confirm the values against your own execution of results.csv before final submission.

### Analysis of the Results

For the successful personal key, Linear Search has the lowest recorded total time at all four dataset sizes in this particular one-query experiment. This is consistent with the fact that Linear Search has no preprocessing cost.

The result does not mean that Linear Search has better asymptotic search complexity than Binary Search or Hash Table Search. It means that, under the measured conditions, the cost of preparing the other data structures exceeded their search-time advantage for the individual query.

As the dataset grows, the recorded total time for Binary Search increases because sorting is included. Hash Table Search also incurs the cost of building its table. These setup costs explain why the total-time comparison differs from the comparison of search-only complexities.

## 10.3 Experimental Results — Unsuccessful Search

The following measurements are recorded for the absent target 192424402.

**Table 10.2: Unsuccessful Search — Recorded Total Time**

| Dataset size | Linear Search total (ms) | Binary Search total (ms) | Hash Search total (ms) |
|--------------|--------------------------|--------------------------|------------------------|
| 1,000        | 0.0279                   | 0.0883                   | 0.0370                 |

|           |         |          |          |
|-----------|---------|----------|----------|
| 10,000    | 0.3266  | 1.0081   | 0.3432   |
| 100,000   | 4.4055  | 13.4602  | 6.2617   |
| 1,000,000 | 32.6264 | 227.2795 | 123.9583 |

## Analysis of the Results

An unsuccessful Linear Search must inspect every record because the target does not appear in the collection. Consequently, its worst-case comparison count is exactly  $\backslash(n\backslash)$ .

Binary Search requires only a logarithmic number of comparisons during its search phase, but the total measured time includes sorting. Hash Table Search requires a table lookup after construction, while the total includes building the table.

For the one-million-record dataset, the unsuccessful Linear Search is slower than its successful counterpart in the recorded measurements. This is consistent with the extra scanning required when the target is absent. However, exact runtime values can vary across executions and systems.

## 10.4 Operation Count Analysis

The recorded successful-search operation counts for the personal key at one million records are:

**Table 10.3: Operation Counts at One Million Records**

| Search condition    | Linear Search | Recursive Binary Search | Hash Table Search |
|---------------------|---------------|-------------------------|-------------------|
| Successful search   | 227,759       | 18                      | 1                 |
| Unsuccessful search | 1,000,000     | 20                      | 1                 |

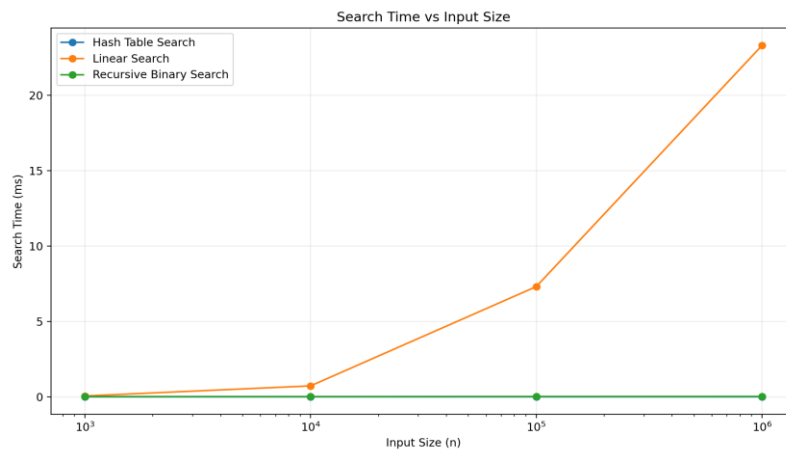
The successful Linear Search does not examine the entire collection because it terminates when the target is found. The unsuccessful search must inspect every element. Binary Search uses a small number of midpoint comparisons, while the benchmark represents Hash Table Search as one logical membership lookup.

Important: These are operation counts, not milliseconds. The hash-table count of one represents the benchmark's logical lookup operation, not every internal operation performed by the Python dictionary.

## 10.5 Graphical Analysis

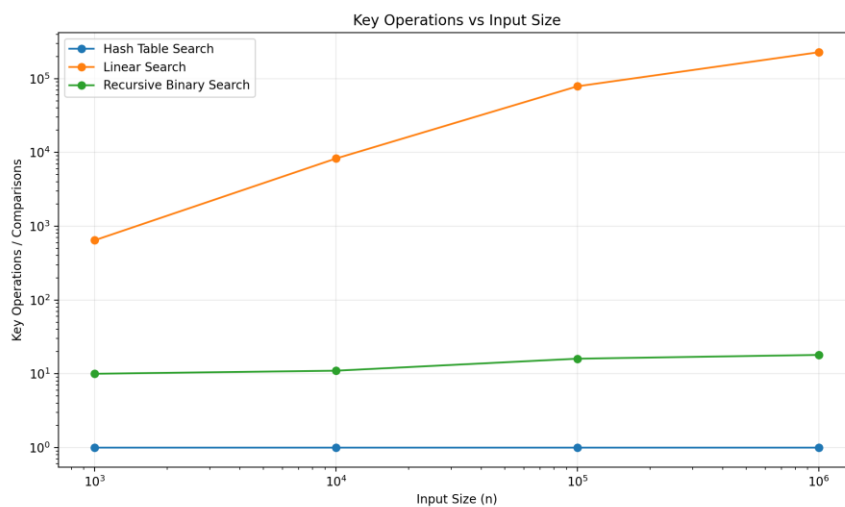
Three graphs are used to visualise the performance differences.

Figure 2: Runtime Comparison



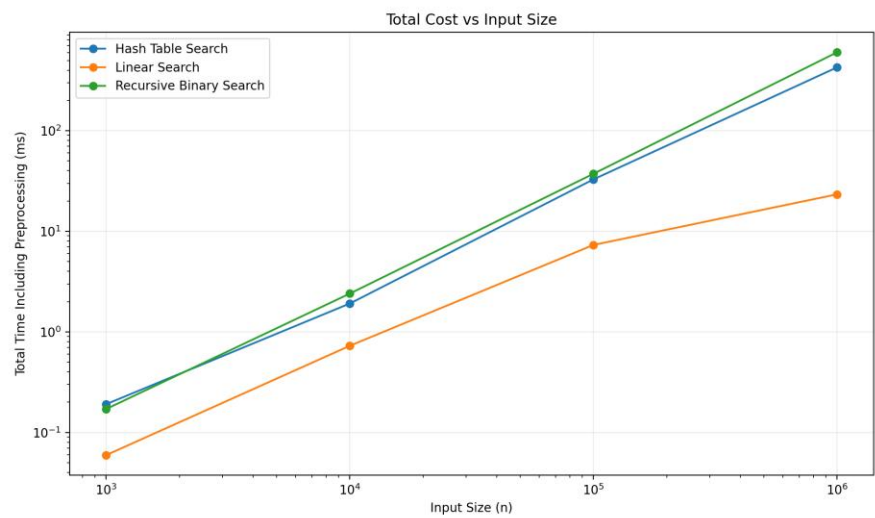
The runtime graph compares the total execution time of the three algorithms across the four dataset sizes. It helps illustrate the impact of dataset growth and preprocessing on the measured end-to-end cost.

Figure 3: Operation Count Comparison



The operation-count graph shows how the recorded number of search operations differs among the algorithms. Linear Search has a growing comparison count, Binary Search uses a logarithmic number of comparisons, and the benchmark models each hash-table query as one logical lookup.

Figure 4: Total Cost Comparison



The total-cost graph illustrates why preprocessing must be included when deciding which algorithm to use. A method with a low search-only complexity can still have a higher total time if its data structure must first be constructed or sorted.

Figure 5: Dataset Benchmark Evidence

```
DAA A1 – Experimental Benchmark – n = 1,000
Name: Jerry David   Reg No: 192424401   Seed: 2401
Timestamp: 2026-10-05 21:34:02 IST

Command: python src/benchmark.py | Dataset Size: 1,000
-----
Target: 192424401 (OWN_REG_NO)
Linear Search      found=True preprocess= 0.0000 ms search= 0.0187 ms total= 0.0187 ms ops=645
Recursive Binary Search found=True preprocess= 0.0789 ms search= 0.0054 ms total= 0.0842 ms ops=10
Hash Table Search  found=True preprocess= 0.0379 ms search= 0.0005 ms total= 0.0384 ms ops=1

Target: 192424402 (OWN_REG_NO_PLUS_1)
Linear Search      found=False preprocess= 0.0000 ms search= 0.0279 ms total= 0.0279 ms ops=1000
Recursive Binary Search found=False preprocess= 0.0849 ms search= 0.0033 ms total= 0.0883 ms ops=10
Hash Table Search  found=False preprocess= 0.0366 ms search= 0.0004 ms total= 0.0370 ms ops=1
```

## DAA A1 – Experimental Benchmark – n = 10,000

Name: Jerry David    Reg No: 192424401    Seed: 2401

Timestamp: 2026-10-05 21:34:02 IST

Command: python src/benchmark.py    |    Dataset Size: 10,000

-----  
Target: 192424401 (OWN\_REG\_NO)

|                         |            |             |           |         |           |        |           |          |
|-------------------------|------------|-------------|-----------|---------|-----------|--------|-----------|----------|
| Linear Search           | found=True | preprocess= | 0.0000 ms | search= | 0.2466 ms | total= | 0.2466 ms | ops=8272 |
| Recursive Binary Search | found=True | preprocess= | 0.9948 ms | search= | 0.0045 ms | total= | 0.9993 ms | ops=11   |
| Hash Table Search       | found=True | preprocess= | 0.4419 ms | search= | 0.0009 ms | total= | 0.4428 ms | ops=1    |

Target: 192424402 (OWN\_REG\_NO\_PLUS\_1)

|                         |             |             |           |         |           |        |           |           |
|-------------------------|-------------|-------------|-----------|---------|-----------|--------|-----------|-----------|
| Linear Search           | found=False | preprocess= | 0.0000 ms | search= | 0.3266 ms | total= | 0.3266 ms | ops=10000 |
| Recursive Binary Search | found=False | preprocess= | 1.0031 ms | search= | 0.0050 ms | total= | 1.0081 ms | ops=13    |
| Hash Table Search       | found=False | preprocess= | 0.3424 ms | search= | 0.0008 ms | total= | 0.3432 ms | ops=1     |

## DAA A1 – Experimental Benchmark – n = 100,000

Name: Jerry David    Reg No: 192424401    Seed: 2401

Timestamp: 2026-10-05 21:34:02 IST

Command: python src/benchmark.py    |    Dataset Size: 100,000

-----  
Target: 192424401 (OWN\_REG\_NO)

|                         |            |             |            |         |           |        |            |           |
|-------------------------|------------|-------------|------------|---------|-----------|--------|------------|-----------|
| Linear Search           | found=True | preprocess= | 0.0000 ms  | search= | 2.5539 ms | total= | 2.5539 ms  | ops=78685 |
| Recursive Binary Search | found=True | preprocess= | 13.9452 ms | search= | 0.0085 ms | total= | 13.9536 ms | ops=16    |
| Hash Table Search       | found=True | preprocess= | 6.6388 ms  | search= | 0.0023 ms | total= | 6.6410 ms  | ops=1     |

Target: 192424402 (OWN\_REG\_NO\_PLUS\_1)

|                         |             |             |            |         |           |        |            |            |
|-------------------------|-------------|-------------|------------|---------|-----------|--------|------------|------------|
| Linear Search           | found=False | preprocess= | 0.0000 ms  | search= | 4.4055 ms | total= | 4.4055 ms  | ops=100000 |
| Recursive Binary Search | found=False | preprocess= | 13.4516 ms | search= | 0.0086 ms | total= | 13.4602 ms | ops=17     |
| Hash Table Search       | found=False | preprocess= | 6.2604 ms  | search= | 0.0013 ms | total= | 6.2617 ms  | ops=1      |

```

DAA A1 – Experimental Benchmark – n = 1,000,000
Name: Jerry David   Reg No: 192424401   Seed: 2401
Timestamp: 2026-10-05 21:34:02 IST

Command: python src/benchmark.py |   Dataset Size: 1,000,000
-----
Target: 192424401 (OWN_REG_NO)
Linear Search      found=True preprocess=  0.0000 ms search=  8.7834 ms total=  8.7834 ms ops=227759
Recursive Binary Search found=True preprocess= 220.6537 ms search=  0.0122 ms total= 220.6659 ms ops=18
Hash Table Search  found=True preprocess= 108.0446 ms search=  0.0022 ms total= 108.0468 ms ops=1

Target: 192424402 (OWN_REG_NO_PLUS_1)
Linear Search      found=False preprocess=  0.0000 ms search= 32.6264 ms total= 32.6264 ms ops=1000000
Recursive Binary Search found=False preprocess= 227.2665 ms search=  0.0130 ms total= 227.2795 ms ops=20
Hash Table Search  found=False preprocess= 123.9557 ms search=  0.0026 ms total= 123.9583 ms ops=1

```

Use actual output captured from your own program execution. The screenshots should correspond to the values reported in your final tables.

## 10.6 Experimental Limitations

The measured times are specific to the execution environment and implementation used for the experiments. They can be affected by CPU performance, memory allocation, operating-system activity, Python implementation details, caching, and the benchmarking method.

Very short operations are particularly sensitive to timing overhead. Therefore, the measured times should be interpreted alongside the operation counts and theoretical complexity, rather than treated as universal performance values.

## 11. TASK 10 — COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

### 11.1 Introduction

Theoretical complexity predicts the growth of an algorithm's work as the input size increases. Experimental analysis measures the behaviour of an actual implementation.

The purpose of this task is to determine whether the recorded observations agree with the theoretical models and to explain any differences between search-only complexity and total execution time.

## 11.2 Linear Search: Theory and Experiment

The theoretical average and worst-case complexity of Linear Search is  $\Theta(n)$ . Its worst-case operation count equals the number of records because every element must be checked when the target is absent.

The experimental results reflect this behaviour. For an unsuccessful search on one million records, the algorithm records 1,000,000 comparisons. For a successful search, the algorithm stops when the target is found, so the comparison count depends on the target's position.

The measured runtime is not necessarily a perfectly straight line as the dataset grows. Runtime depends on the location of the target, the programming environment, and other implementation factors. Nevertheless, the operation-count model agrees with the theoretical linear growth.

## 11.3 Binary Search: Theory and Experiment

Binary Search has  $\Theta(\log n)$  search complexity because each recursive call reduces the remaining interval by approximately half.

The experimental operation counts are consistent with this property. For the successful one-million-record query, only 18 search operations are recorded, and the unsuccessful query records 20.

However, the recorded total time is considerably larger than the search-only complexity might suggest. This is because the benchmark includes the cost of sorting the dataset before the query.

Thus, the experiment demonstrates that the complexity of the search phase and the complexity of the complete operation are not necessarily the same.

## 11.4 Hash Table Search: Theory and Experiment

Under the expected hashing model, hash-table lookup has  $\Theta(1)$  complexity. The benchmark represents the membership test as one logical lookup for both successful and unsuccessful queries.

The experimental results also show that the total time includes table construction. For a single query, the initial cost of creating the table may be greater than the cost of performing a direct sequential scan.

When many queries use the same table, construction can be amortised across those queries. This changes the practical suitability of the algorithm.

**Table 11.1: Theoretical and Experimental Comparison**

| Algorithm         | Theoretical expectation        | Experimental observation                 | Explanation                                             |
|-------------------|--------------------------------|------------------------------------------|---------------------------------------------------------|
| Linear Search     | Linear worst-case comparisons  | Absent-key search examines all records   | Every record must be checked                            |
| Binary Search     | Logarithmic search comparisons | Few midpoint comparisons                 | The interval is halved repeatedly                       |
| Hash Table Search | Expected constant-time lookup  | One logical lookup is recorded per query | The benchmark counts membership lookup as one operation |

### 11.5 Why Measured Time Differs from Theoretical Complexity

Theoretical complexity abstracts away several practical factors:

- Constant factors associated with individual operations.
- Memory allocation and data-structure construction.
- CPU caching and memory access patterns.
- Interpreter and library implementation overhead.
- Background processes and operating-system scheduling.
- Timing precision and measurement overhead.
- The target's position or absence in the dataset.

Big-O and Big-Theta notation describe growth rates, not exact execution times. Therefore, two algorithms with different asymptotic complexities can show unexpected runtime relationships on small inputs or when setup costs dominate.

### 11.6 Conclusion of the Comparison

The operation counts support the theoretical distinction among the algorithms: Linear Search may inspect every record, Binary Search uses logarithmic search steps, and Hash Table Search performs an expected constant-time lookup after construction.

The total-time measurements show that preprocessing must be included in a fair comparison. The experiments therefore support a workload-dependent design decision rather than selecting an algorithm solely from its search-only Big-O notation.

## **12. TASK 11 — RECOMMEND AND DEFEND THE FINAL DESIGN**

### **12.1 Evaluation Criteria**

The algorithm should be selected according to the number of records, number of queries, frequency of updates, preprocessing cost, memory availability, and expected response time.

A method that is suitable for one search may not be suitable for thousands of searches against the same dataset. The final recommendation must therefore consider both the original workload and the revised workload introduced in Task 12.

### **12.2 Recommendation for the Original Single-Query Workload**

For a single lookup on an unsorted dataset, Linear Search is a reasonable baseline. It does not require sorting or index construction, so the system can begin searching immediately.

The successful-query benchmark recorded lower total times for Linear Search than for the other two algorithms at all four dataset sizes. The result supports using Linear Search for the measured one-query baseline, although its performance depends on the target position.

Binary Search is not automatically the best choice for a single query because sorting the dataset may dominate the total cost. Hash Table Search also incurs construction overhead if a reusable table has not already been created.

### **12.3 Recommendation for Repeated Searches**

For a system that performs many searches on the same dataset, a reusable index is preferable to repeatedly scanning the records.

Binary Search is suitable when the data is already sorted and that order can be maintained. It offers logarithmic search time, but changes to the data may require maintaining the sorted structure.

Hash Table Search is particularly attractive when registration-number lookups are frequent and updates are relatively infrequent. The hash table can be built once and reused for many queries, reducing the average construction cost per query.

**Table 12.1: Algorithm Selection by Workload**

| Workload                                 | Recommended approach            | Reason                                           |
|------------------------------------------|---------------------------------|--------------------------------------------------|
| One query on an unsorted dataset         | Linear Search as a baseline     | No preprocessing required                        |
| Repeated queries on sorted data          | Binary Search                   | Low search cost when sorted data can be reused   |
| Frequent queries with infrequent updates | Reusable Hash Table index       | Expected constant-time lookup after construction |
| Small dataset with occasional queries    | Linear Search may be sufficient | Simplicity and low setup cost                    |

## 12.4 Trade-Off Analysis

**Table 12.2: Algorithm Trade-Off Comparison**

| Criterion             | Linear Search                  | Binary Search                         | Hash Table Search                             |
|-----------------------|--------------------------------|---------------------------------------|-----------------------------------------------|
| Initial setup         | Minimal                        | Sorting required if input is unsorted | Table construction required                   |
| Lookup growth         | Linear                         | Logarithmic                           | Expected constant                             |
| Additional memory     | Low                            | Depends on sorting and implementation | Higher because an index is stored             |
| Repeated queries      | Inefficient for large datasets | Efficient if sorted data is reusable  | Highly suitable for read-heavy workloads      |
| Update considerations | No index maintenance           | Sorted order must be preserved        | Index must remain consistent with the dataset |

## 12.5 Final Recommendation

For the original single-query experiment, Linear Search is a defensible choice because it avoids preprocessing and produced the lowest measured total times in the recorded successful-query benchmark.

For a high-frequency student-record retrieval system, the recommendation changes. A reusable hash table is a stronger design when the system serves many searches against a dataset that changes infrequently.

This recommendation is supported by the Task-12 workload, which evaluates 50,000 searches on one million records. The final design should therefore be chosen according to the actual workload rather than assuming that one algorithm is universally optimal.

### 13. TASK 12 — DAY-4 DESIGN CHALLENGE

#### 13.1 A. Impact Analysis

The revised system requirement is:

- One million student records.
- 50,000 search requests per hour.
- New records are added only once per day.

This change significantly affects the design decision. The system is no longer concerned with a single lookup. It must serve a large number of requests while avoiding unnecessary repeated work.

The low update frequency is important because it allows a data structure to be constructed once and reused. Instead of rebuilding an index for every query, the system can prepare the current dataset and then serve many searches against that structure.

The key design objective is therefore to reduce the cumulative cost of repeated searches.

**Table 13.1: Revised System Requirements**

| Requirement       | Value                                                 |
|-------------------|-------------------------------------------------------|
| Number of records | 1,000,000                                             |
| Search frequency  | 50,000 searches per hour                              |
| Update frequency  | Once per day                                          |
| Main concern      | Efficient repeated lookup                             |
| Selected design   | Reusable Hash Table index                             |
| Update approach   | Refresh or rebuild the index when the dataset changes |

## **13.2 B. Evaluation of the Original Design**

### **Linear Search**

Linear Search is not a suitable primary strategy for this revised workload. Every unsuccessful query requires examining all one million records. Even when the target is present, a large number of records may need to be examined before it is found.

If the system performs 50,000 searches per hour, repeatedly scanning a large dataset can create substantial cumulative work.

### **Recursive Binary Search**

Binary Search provides logarithmic search performance once the data is sorted. Its search phase is therefore efficient for a million records.

However, the system must maintain sorted data or account for sorting when preparing the dataset. Since new records are added only once per day, sorting or maintaining an ordered structure may be feasible, but the preparation cost and update process must still be considered.

### **Hash Table Search**

Hash Table Search requires the dataset to be indexed before lookups begin. Once the hash table has been constructed, it can be reused for the repeated registration-number queries.

This approach aligns with the revised requirement because searches occur frequently while updates occur infrequently.

## **13.3 C. Adaptation or Redesign**

The selected response is to redesign the system around a reusable Hash Table index.

The proposed workflow is:

1. Load the latest student-record dataset.
2. Construct the hash table using registration numbers as keys.
3. Keep the index available while the dataset remains unchanged.
4. Process incoming search requests using the existing hash table.
5. Update or rebuild the index when the daily dataset changes.
6. Validate the refreshed index before using it for subsequent requests.

The expected complexity of index construction is:

$$T_{\text{build}}(n) = \Theta(n)$$

The expected lookup complexity is:

$$T_{\text{lookup}}(n) = \Theta(1)$$

For  $q$  searches against the same index, the expected combined cost can be represented as:

$$T(n, q) = \Theta(n) + q\Theta(1)$$

Therefore,

$$T(n, q) = \Theta(n + q)$$

This model explains why the initial construction cost can become relatively less significant as the number of queries increases.

#### 13.4 D. Evidence-Based Defence

The dedicated Task-12 benchmark uses one million records and 50,000 deterministic searches. The recorded results are:

**Table 13.2: Task-12 Benchmark Comparison**

| Algorithm               | Preprocessing<br>(ms) | 50,000-search phase<br>(ms) | Total (ms) |
|-------------------------|-----------------------|-----------------------------|------------|
| Recursive Binary Search | 251.7434              | 194.9496                    | 446.6929   |
| Hash Table Search       | 135.1444              | 12.6440                     | 147.7883   |

Use these figures only if they match the Task-12 results produced by your supplied project.

#### Interpretation

The total recorded time for Hash Table Search is 147.7883 ms, compared with 446.6929 ms for Recursive Binary Search.

The ratio is:

$$\text{Time ratio} = \frac{446.6929}{147.7883} \approx 3.02$$

Therefore, the hash-based implementation is approximately 3.02 times faster in this recorded benchmark.

The percentage of Binary Search's total time used by Hash Table Search is:

$$\frac{147.7883}{446.6929} \times 100 \approx 33.1\%$$

Thus, Hash Table Search uses approximately 33.1% of the measured Binary Search total in this run.

This comparison includes preprocessing, so it represents the recorded end-to-end benchmark rather than lookup-only performance.

### 13.5 E. Final Decision

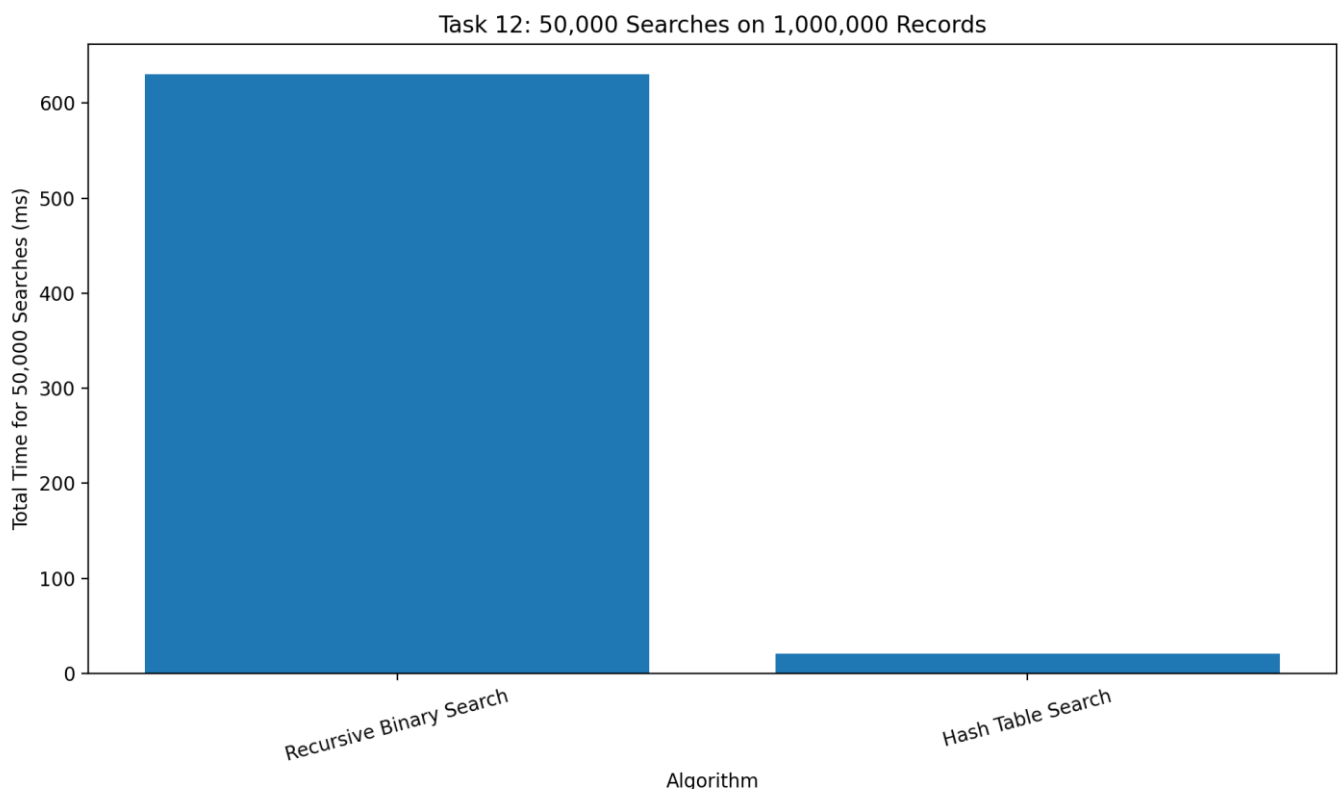
The final decision for Task 12 is REDESIGN.

A reusable hash table is appropriate because the system contains one million records, performs 50,000 searches per hour, and updates its data only once per day. The index can be constructed once and reused across many searches, while the daily update process can refresh the structure when the dataset changes.

The benchmark also supports this decision: the recorded total for Hash Table Search is lower than that of Recursive Binary Search in the 50,000-search experiment.

The redesign is therefore supported by the workload requirements, the expected complexity of repeated lookup, and the measured benchmark results.

Figure 6: Task-12 Benchmark



```
PS C:\Users\Jerry David\OneDrive\문서\DAA A1\192424401-JerryDavid> python src/benchmark.py

Target: 192424401 (OWN_REG_NO)
Linear Search      found=True  preprocess=0.0000 ms  search=0.7271 ms  total=0.7271 ms  ops=8272
Recursive Binary Search found=True  preprocess=2.3874 ms  search=0.0158 ms  total=2.4032 ms  ops=11
Hash Table Search  found=True  preprocess=1.9110 ms  search=0.0025 ms  total=1.9135 ms  ops=1

Target: 192424402 (OWN_REG_NO_PLUS_1)
Linear Search      found=False preprocess=0.0000 ms  search=1.0632 ms  total=1.0632 ms  ops=10000
Recursive Binary Search found=False preprocess=2.4232 ms  search=0.0126 ms  total=2.4358 ms  ops=13
Hash Table Search  found=False preprocess=1.7045 ms  search=0.0062 ms  total=1.7107 ms  ops=1

Dataset Size: 100,000

-----

Target: 192424401 (OWN_REG_NO)
Linear Search      found=True  preprocess=0.0000 ms  search=7.3150 ms  total=7.3150 ms  ops=78685
Recursive Binary Search found=True  preprocess=37.2895 ms  search=0.0220 ms  total=37.3115 ms  ops=16
Hash Table Search  found=True  preprocess=32.8303 ms  search=0.0034 ms  total=32.8337 ms  ops=1

Target: 192424402 (OWN_REG_NO_PLUS_1)
Linear Search      found=False preprocess=0.0000 ms  search=11.2347 ms  total=11.2347 ms  ops=100000
Recursive Binary Search found=False preprocess=42.8757 ms  search=0.0220 ms  total=42.8977 ms  ops=17
Hash Table Search  found=False preprocess=28.8179 ms  search=0.0026 ms  total=28.8205 ms  ops=1

Dataset Size: 1,000,000

-----

Target: 192424401 (OWN_REG_NO)
Linear Search      found=True  preprocess=0.0000 ms  search=23.3140 ms  total=23.3140 ms  ops=227759
Recursive Binary Search found=True  preprocess=602.7269 ms  search=0.0256 ms  total=602.7525 ms  ops=18
Hash Table Search  found=True  preprocess=426.9459 ms  search=0.0041 ms  total=426.9500 ms  ops=1

Target: 192424402 (OWN_REG_NO_PLUS_1)
Linear Search      found=False preprocess=0.0000 ms  search=87.3863 ms  total=87.3863 ms  ops=1000000
Recursive Binary Search found=False preprocess=612.6587 ms  search=0.0514 ms  total=612.7101 ms  ops=20
Hash Table Search  found=False preprocess=583.3313 ms  search=0.0029 ms  total=583.3342 ms  ops=1

=====
TASK 12 - 50,000 SEARCHES ON 1,000,000 RECORDS
=====
Recursive Binary Search  search=630.6110 ms  ops=973840  found=25000/50000
Hash Table Search        search=21.0164 ms  ops=50000   found=25000/50000
```

The screenshot should show the actual 50,000-search experiment, its recorded results, and the required student identification and system date/time.

## 14. CONCLUSION

This project investigated the design and analysis of three searching algorithms for a student-record retrieval system: Linear Search, Recursive Binary Search, and Hash Table Search.

The theoretical analysis showed that Linear Search has linear average and worst-case search complexity. Binary Search has logarithmic search complexity when the input is sorted, while Hash Table Search provides expected constant-time lookup after the table has been constructed.

The experiments demonstrated that search-only complexity is not sufficient to determine the fastest approach for a single query. Linear Search avoids preprocessing, whereas Binary Search requires sorting and Hash Table Search requires table construction. In the recorded successful-query experiment, Linear Search achieved the lowest total time at each tested dataset size.

The results also showed the importance of considering the target's presence or absence. An unsuccessful Linear Search must inspect every record, while a successful search can terminate as soon as the target is found. Binary Search and Hash Table Search use different structures to reduce the work required during the lookup phase.

The Task-12 requirement introduced a different workload: one million records, 50,000 searches per hour, and only one update per day. Under these conditions, a reusable hash table becomes a more appropriate design because its construction cost can be reused across many requests. The recorded benchmark showed a total of 147.7883 ms for Hash Table Search compared with 446.6929 ms for Recursive Binary Search.

**Table 14.1: Final Findings**

| Evaluation criterion                                                          | Finding                                                                              |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Simplest implementation                                                       | Linear Search                                                                        |
| Lowest search-only growth among the three methods requiring search operations | Binary Search, with $\Theta(\log n)$ search complexity                               |
| Expected constant-time lookup                                                 | Hash Table Search                                                                    |
| Strong baseline for one query on unsorted data                                | Linear Search                                                                        |
| Recommended design for the revised high-frequency workload                    | Reusable Hash Table index                                                            |
| Main lesson                                                                   | Preprocessing and query frequency must be considered alongside asymptotic complexity |

In conclusion, the project demonstrates that efficient algorithm design requires both mathematical analysis and practical evaluation. The most appropriate strategy depends on dataset size, preprocessing requirements, query frequency, memory usage, and update frequency. Linear Search provides a simple single-query baseline, Binary Search is effective on ordered data, and a reusable Hash Table index is a strong choice for the high-frequency workload specified in Task 12.

## 15. APPENDIX — REPRODUCIBILITY AND SUBMISSION

### 15.1 Execution Procedure

The project is implemented in Python. Run the following commands from the project root directory.

Step 1: Install the dependencies

```
python -m pip install -r requirements.txt
```

Step 2: Generate the datasets

```
python src/dataset_generator.py
```

Step 3: Run the correctness tests

```
python src/correctness_test.py
```

Step 4: Run the benchmark experiments

```
python src/benchmark.py
```

Step 5: Generate the graphs

```
python src/plot_results.py
```

Run these commands from the correct project directory. If a command fails, check that Python is installed, the required dependencies are available, and the file paths match the extracted project structure.

## 15.2 Screenshot Checklist

Insert your own execution screenshots for the following:

- Correctness-testing output.
- Benchmark for 1,000 records.
- Benchmark for 10,000 records.
- Benchmark for 100,000 records.
- Benchmark for 1,000,000 records.
- Task-12 experiment using 50,000 searches.

Each screenshot should show authentic output from your program. Ensure that the student identification and system date/time are visible if required by your assessment instructions.

Do not fabricate execution results or present a prepared graph as a terminal-execution screenshot. The screenshots should demonstrate that you actually ran the implementation.

## 15.3 Final Submission Checklist

| No. | Requirement                                                          | Status |
|-----|----------------------------------------------------------------------|--------|
| 1   | Cover page contains the correct student name and registration number | Done   |
| 2   | All twelve tasks are included                                        | Done   |
| 3   | Pseudocode and mathematical derivations are present                  | Done   |
| 4   | Best, average, and worst case analyses are included                  | Done   |

|    |                                                                                   |      |
|----|-----------------------------------------------------------------------------------|------|
| 5  | Binary Search recurrence and its solution are explained                           | Done |
| 6  | Correctness-testing results are documented                                        | Done |
| 7  | Benchmark tables match the generated CSV                                          | Done |
| 8  | All required graphs are included                                                  | Done |
| 9  | Task-12 analysis and final redesign recommendation are included                   | Done |
| 10 | Actual screenshots have replaced every screenshot placeholder                     | Done |
| 11 | The final report is exported to PDF                                               | Done |
| 12 | Source code, datasets, results, and documentation are included in the project ZIP | Done |
| 13 | The final GitHub repository link is included in the report                        | Done |